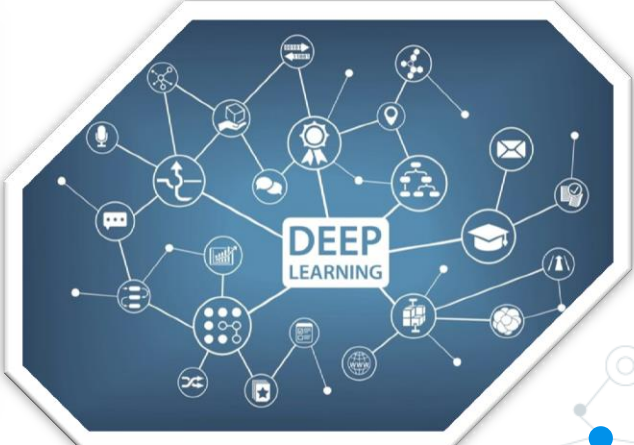
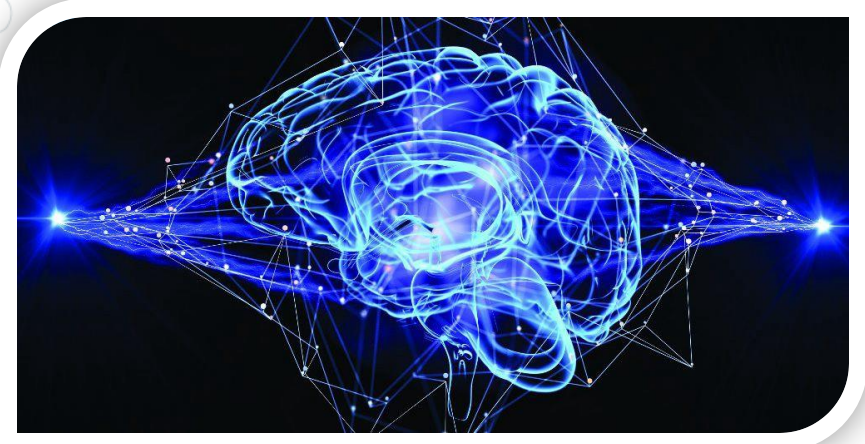


DEEP LEARNING



HISTORY

ARTIFICIAL INTELLIGENCE

Early artificial intelligence stirs excitement.



MACHINE LEARNING

Machine learning begins to flourish.



DEEP LEARNING

Deep learning breakthroughs drive AI boom.



1950's

1960's

1970's

1980's

1990's

2000's

2010's

HISTORY

- 1943: First artificial neural networks (McCulloch, Pitts)
- 1956: First conference defining “AI” in Dartmouth (Shannon, Minsky etc.)
- 1986: Training multi-layer neural nets using backpropagation (Hinton, Rumelhart, Williams)
- 1989: Convolutional neural networks (LeCun)
- 1997: Deep blue beats Kasparov in chess
- **2009**: Big bang of deep learning, use of GPUs for accelerated training
- 2012: Numerous competitions won with superhuman performance in computer vision using deep CNNs (Krizhevsky, Ciresan etc.)
- 2016: AlphaGo beats Lee Sedol in Go

HISTORY

THE EXPANDING UNIVERSE OF MODERN AI

"THE BIG BANG"

Big Data
GPU
Algorithms

Berkeley
Carnegie Mellon University

DEEP MIND
MIT
NYU

OpenAI
Université de Montréal
UNIVERSITY OF OXFORD
UNIVERSITY OF TORONTO

RESEARCH

CORE TECHNOLOGY / FRAMEWORKS

facebook torch Université de Montréal theano
Google TensorFlow Berkeley Caffe
Microsoft CNTK UNIVERSITY OF OXFORD
NVIDIA cuDNN

Preferred Networks Chainer

AI-as-a-PLATFORM

amazon web services
IBM Watson
Google
Microsoft Azure

START-UPS

api.ai Personal Assistants conversational interface
drive.ai Automotive computer vision
nervana Tech AI-as-a-service
SADAKO Waste Management smart robotics
MetaMind eCommerce & Medical recommendation engine
SocialEyes Medical diabetic retinopathy
clarifai Tech computer vision
Morpho Tech computer vision
Orbital Insight Geospatial predictions from images
deep genomics Genomics genetic interpretation

1,000+ AI START-UPS

\$5B IN FUNDING

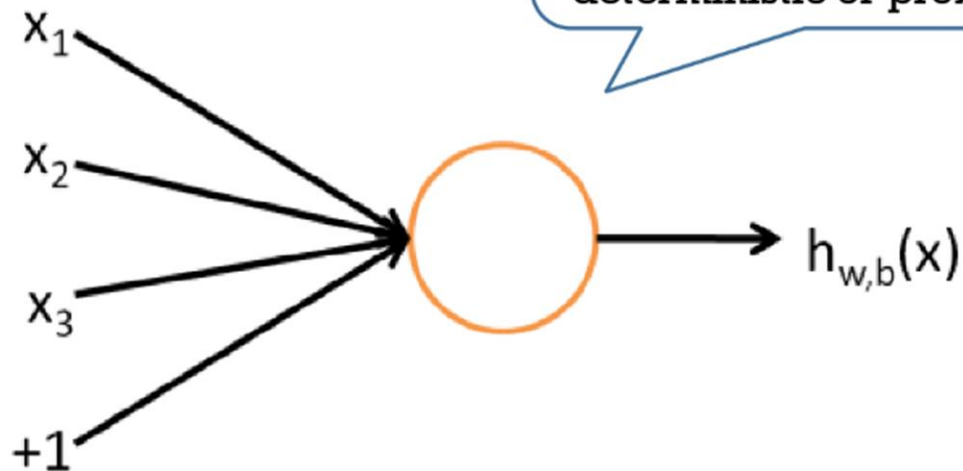
Source: Venture Scanner

INDUSTRY LEADERS

SIEMENS
Alibaba.com
AstraZeneca
Audi
Baidu 百度
Bloomberg
charles SCHWAB
cisco
ebay
FANUC ROBOTICS
Ford
GE
gsk
Libby
MARIA CHURCH'S GENERAL HOSPITAL
Mercedes-Benz
MERCK
Pinterest
Schlumberger
Shell
TARGET
TESLA
TOYOTA
Twitter
UBER
VOLVO
Walmart
YAHOO
Yandex
yelp

NEURON

Perceptron.

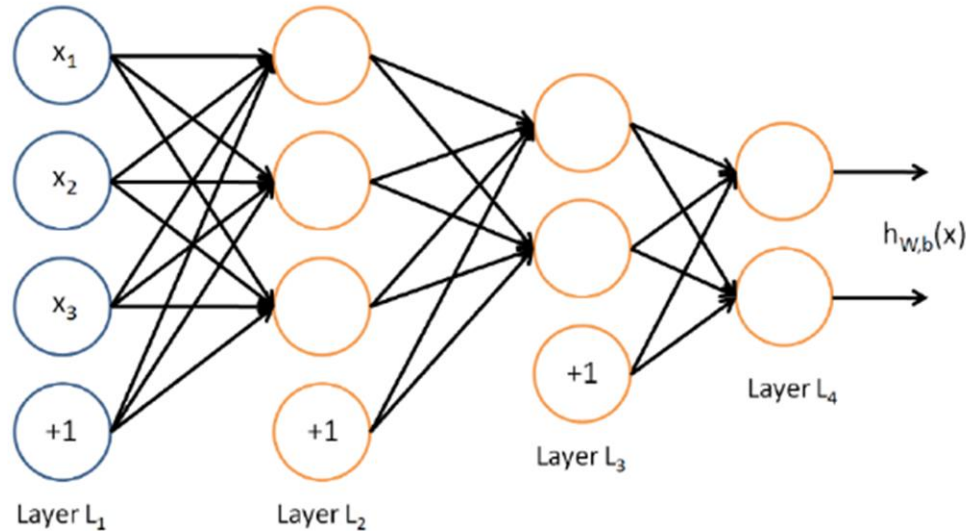


Depending on function f ,
neurons can be:
real-valued or binary-valued;
deterministic or probabilistic.

$$h_{w,b}(x) = f(w^T x) = f\left(\sum_{i=1}^d w_i x_i + b\right)$$

WHAT IS DEEP LEARNING

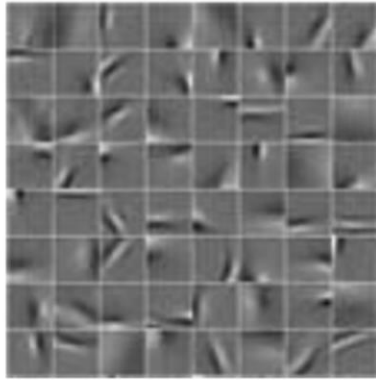
Biologically-inspired multilayer neural networks



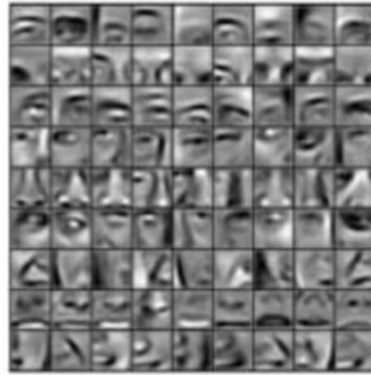
Both supervised and unsupervised

WHAT IS DEEP LEARNING

Example. Face recognition (Facebook)



Edges



Eyes, Noses, Mouths



Faces

Deeper layers learn higher-order features

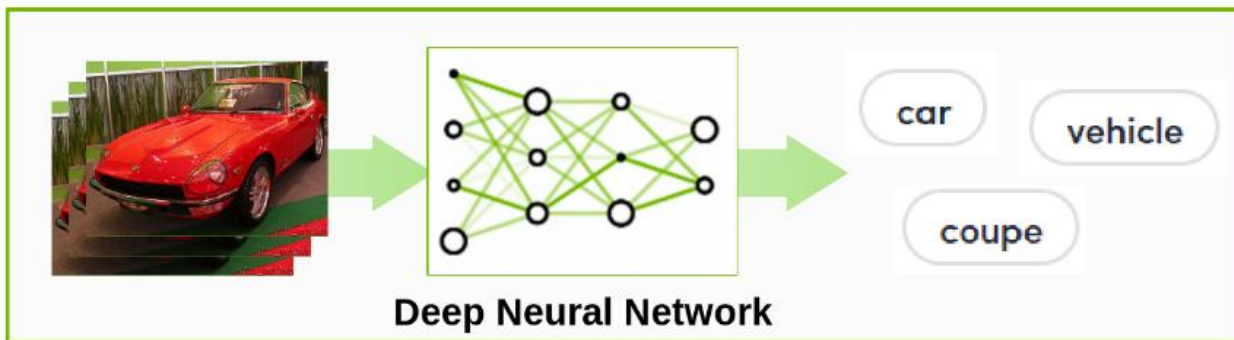
A NEW COMPUTER MODELLING

Algorithms that Learn from Examples



Traditional Approach

- Requires domain experts
- Time consuming
- Error prone
- Not scalable to new problems

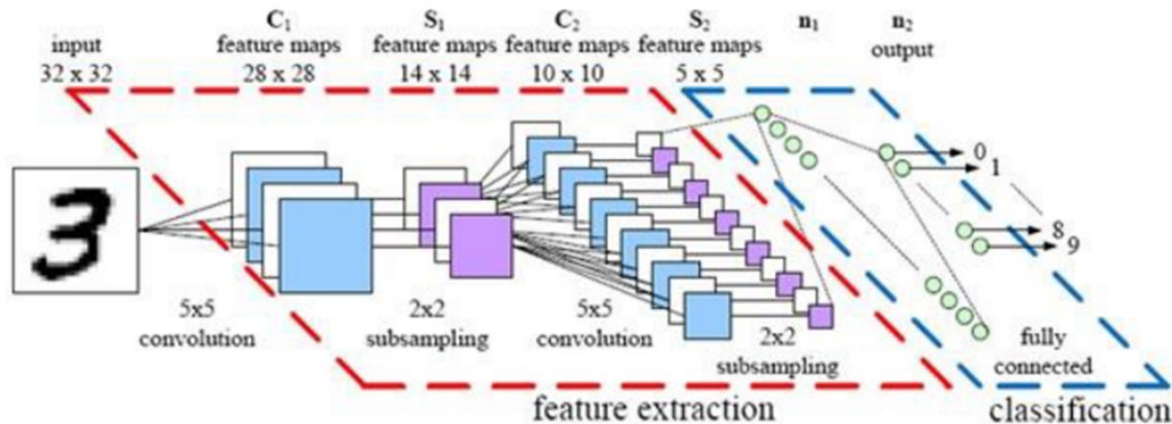


Deep Learning Approach

- ✓ Learn from data
- ✓ Easily to extend
- ✓ Speedup with GPUs

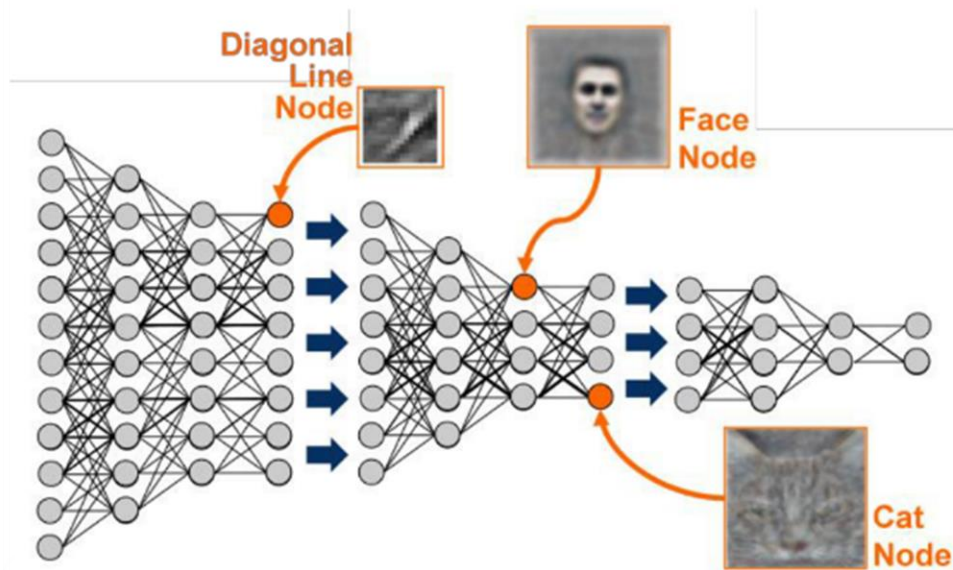
APPLICATIONS

HANDWRITING RECOGNITION



APPLICATIONS

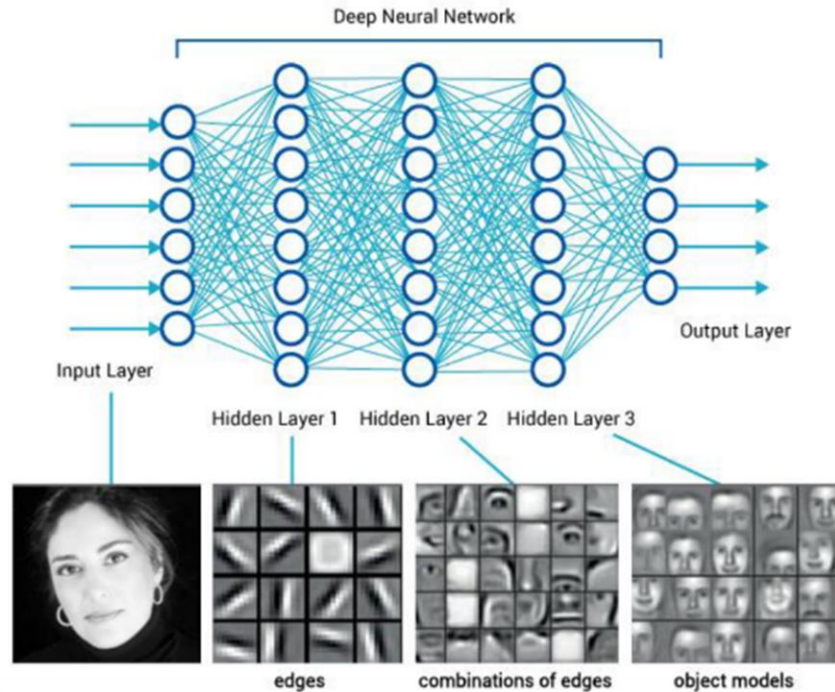
GOOGLE CAT VIDEOS



Deep Learning
with GPUs

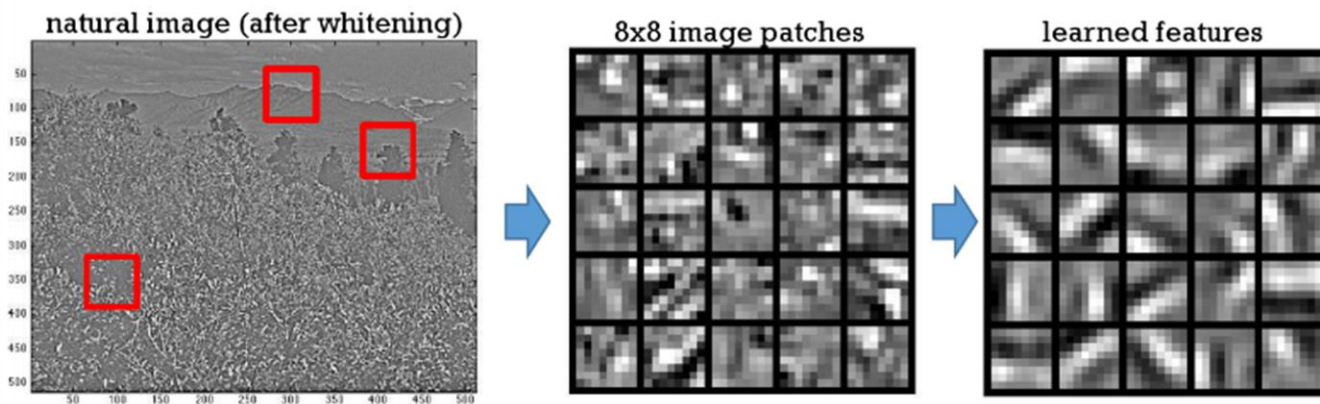
APPLICATIONS

FACE RECOGNITION



APPLICATIONS

NATURAL IMAGES



Edge features similar to those from neuroscience experiments (see Hubel & Wiesel Cat Experiment, 1959).

APPLICATIONS

SPEECH TRANSLATION



From Hidden Markov Models to Recurrent Neural Networks

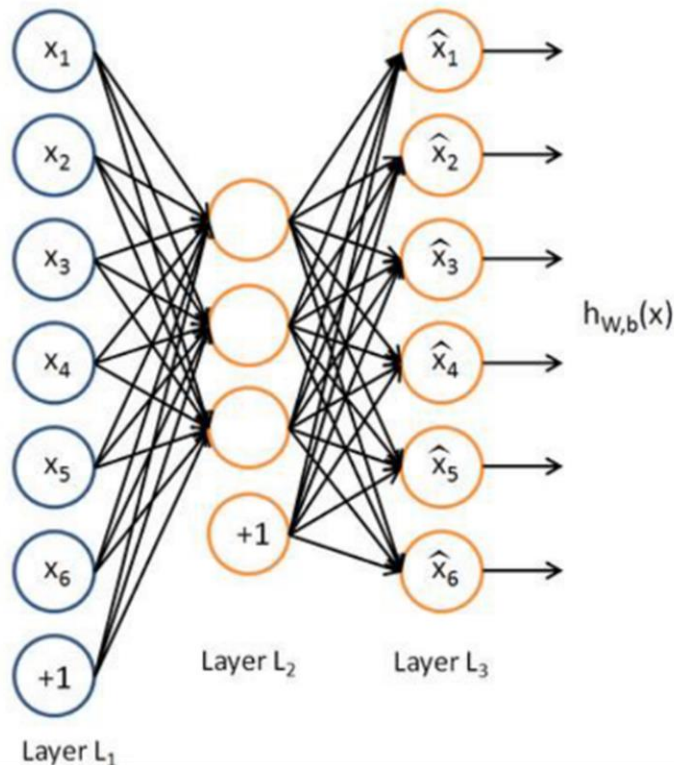
APPLICATIONS

AUTOENCODERS

Training a multilayer neural network to reconstruct the input from a **reduced representation**.

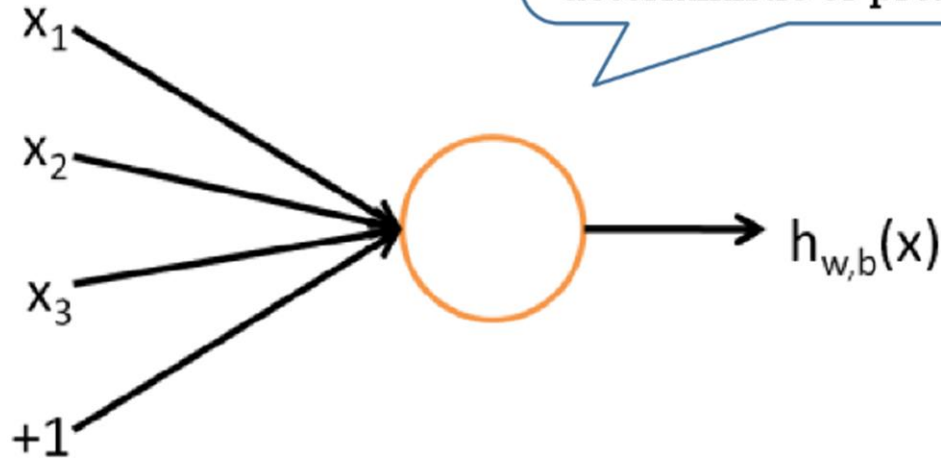
Strategies for Dimensionality Reduction

- Few hidden neurons
- Sparse activations



FEED FORWARD NETWORKS

Perceptron.

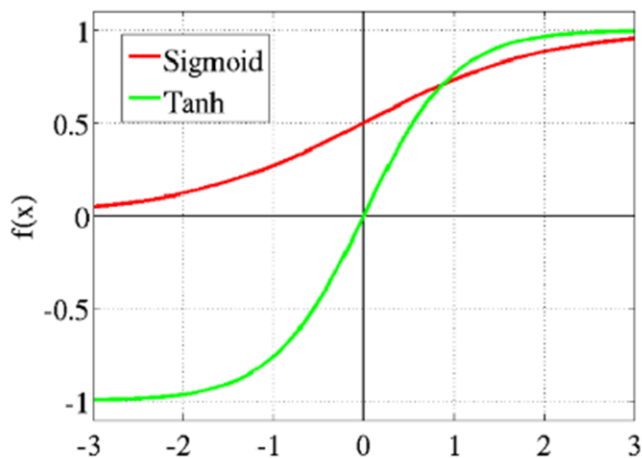


Depending on function f ,
neurons can be:
real-valued or binary-valued;
deterministic or probabilistic.

$$h_{w,b}(x) = f(w^T x) = f\left(\sum_{i=1}^d w_i x_i + b\right)$$

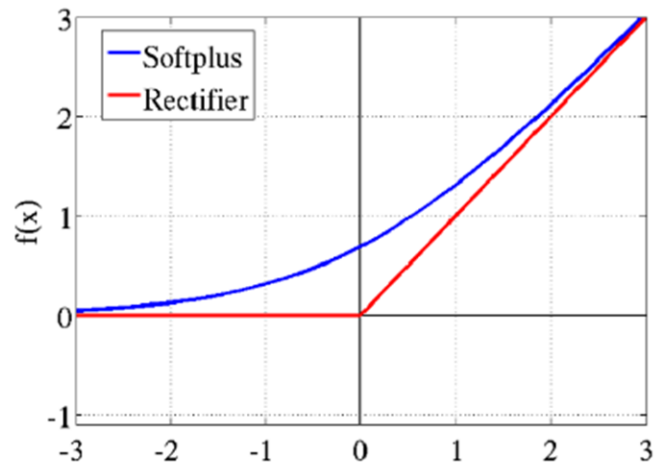
FEED FORWARD NETWORKS

ACTIVATION FUNCTIONS



$$\text{sigmoid } f(z) = \frac{1}{1+e^{-z}}$$

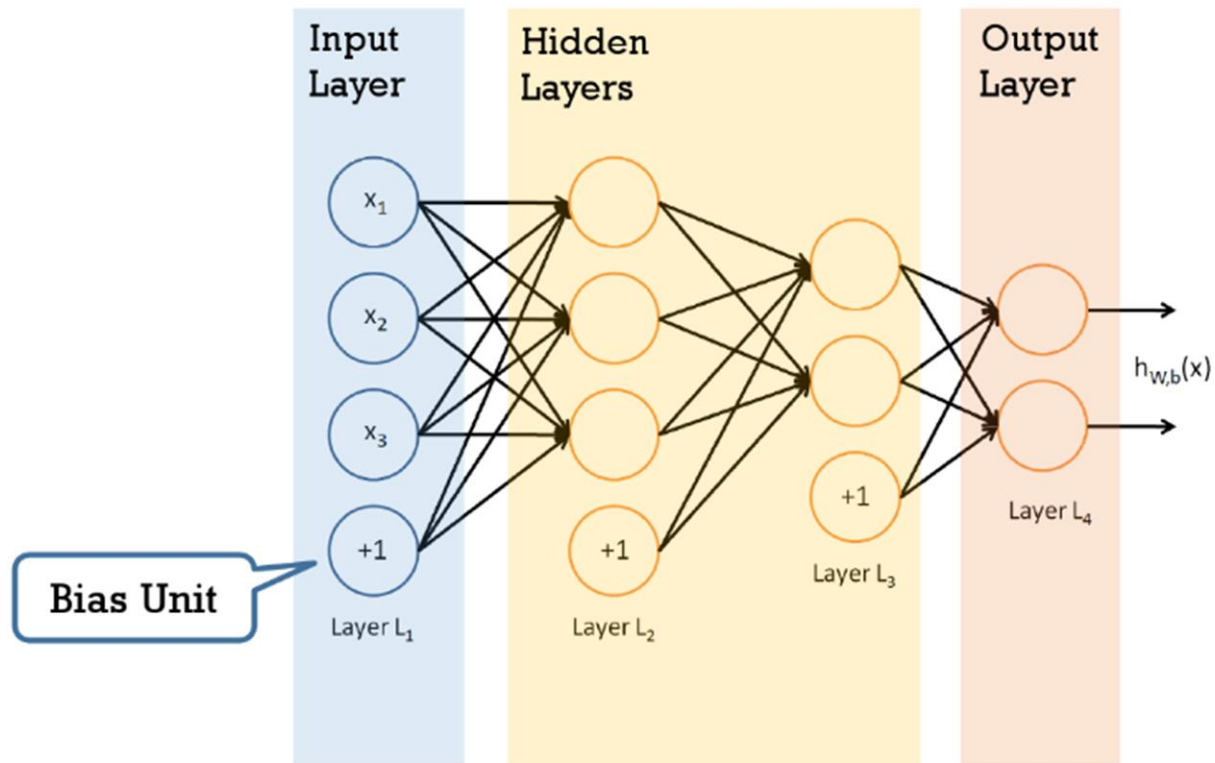
$$\text{tanh } f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$



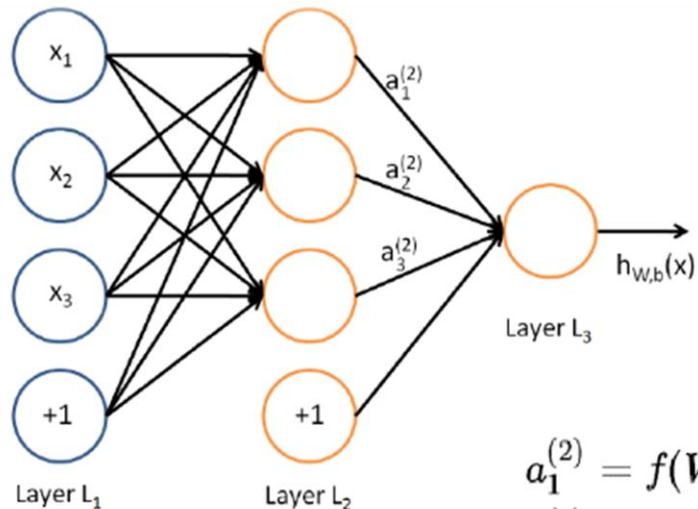
$$\text{softplus } f(z) = \ln(1 + e^z)$$

$$\text{rectified linear unit (ReLU)} \quad f(z) = \max(0, z)$$

MULTI-LAYERED NETWORKS

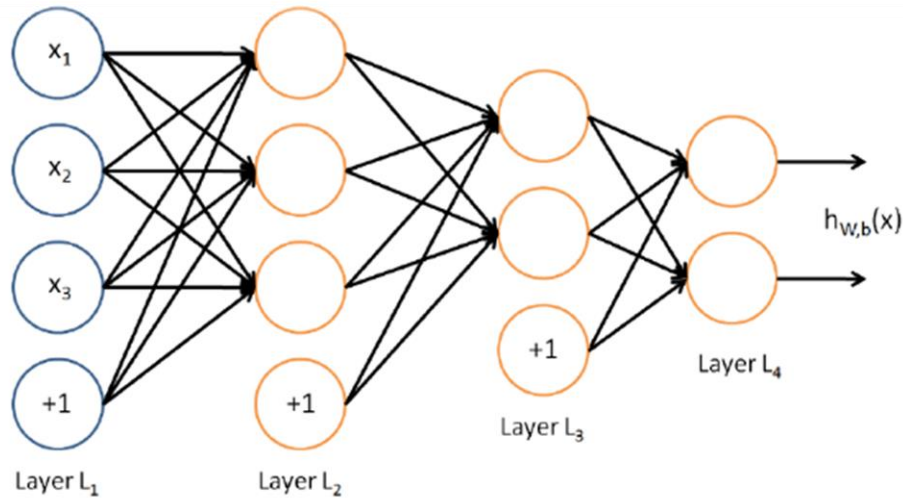


MULTI-LAYERED NETWORKS



$$\begin{aligned}a_1^{(2)} &= f(W_{11}^{(1)} x_1 + W_{12}^{(1)} x_2 + W_{13}^{(1)} x_3 + b_1^{(1)}) \\a_2^{(2)} &= f(W_{21}^{(1)} x_1 + W_{22}^{(1)} x_2 + W_{23}^{(1)} x_3 + b_2^{(1)}) \\a_3^{(2)} &= f(W_{31}^{(1)} x_1 + W_{32}^{(1)} x_2 + W_{33}^{(1)} x_3 + b_3^{(1)}) \\h_{W,b}(x) &= a_1^{(3)} = f(W_{11}^{(2)} a_1^{(2)} + W_{12}^{(2)} a_2^{(2)} + W_{13}^{(2)} a_3^{(2)} + b_1^{(2)})\end{aligned}$$

MULTI-LAYERED NETWORKS



Feedforward Neural Network.

Neural Network Architecture.

Arrangement of neurons, e.g. number of neurons in each layer.

$$z^{(l+1)} = W^{(l)} a^{(l)} + b^{(l)}$$
$$a^{(l+1)} = f(z^{(l+1)})$$

Activation

UNIVERSAL APPROXIMATION

Theorem (Universal approximation theorem)

Let $\mathcal{X}$ be a closed and bounded subset of $\mathbb{R}^m$, and f be any continuous function on $\mathcal{X}$. Assume the activation function $\phi : \mathbb{R} \rightarrow \mathbb{R}$ is non-constant, bounded and continuous. Then given any $\epsilon > 0$, there exist N , b_i , v_i and vectors $w_i \in \mathbb{R}^m$ such that when we define

$$F(x) = \sum_{i=1}^N v_i \phi(\langle w_i, x \rangle + b_i),$$

we have

$$|F(x) - f(x)| < \epsilon$$

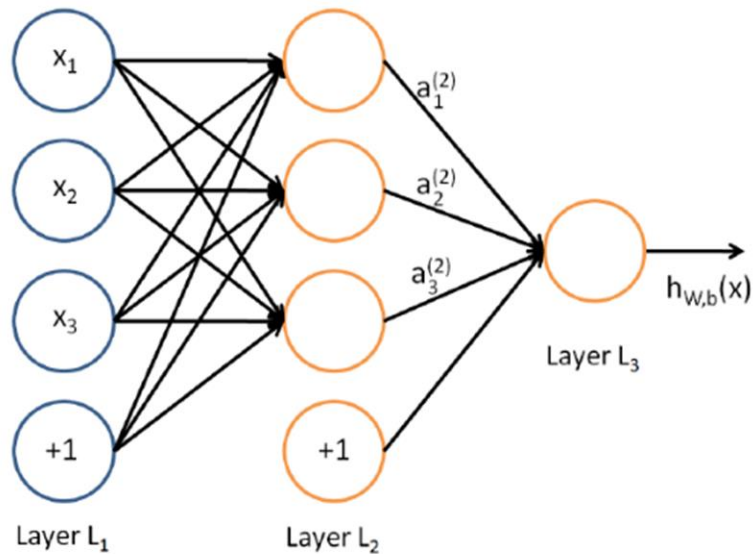
for all $x \in \mathcal{X}$.

BACK PROPOGATION

The feed forward component is designed to give an estimate $\mathbf{h}(\mathbf{x})$ of the output given the input $\mathbf{x}$.

The back propagation part is to adjust the parameters $\mathbf{W}$ and $\mathbf{b}$ using gradient descent on the loss function of $\mathbf{y}$ and $\mathbf{h}(\mathbf{x})$.

BACK PROPAGATION



Forward Propagation.

$$z^{(2)} = W^{(1)}x + b^{(1)}$$

$$a^{(2)} = f(z^{(2)})$$

$$z^{(3)} = W^{(2)}a^{(2)} + b^{(2)}$$

$$h_{W,b}(x) = a^{(3)} = f(z^{(3)})$$

BACK PROPAGATION

Chain Rule for Neural Networks.

$$\begin{aligned}\frac{\partial}{\partial W_{ij}^{(l)}} \left(\frac{1}{2} \|a^{(n_l)} - y\|^2 \right) &= (a^{(n_l)} - y) \frac{\partial}{\partial W_{ij}^{(l)}} f(z^{(n_l)}) \\&= (a^{(n_l)} - y) f'(z^{(n_l)}) \frac{\partial}{\partial W_{ij}^{(l)}} (W^{(n_l-1)} a^{(n_l-1)} + b^{(n_l-1)}) \\&= (a^{(n_l)} - y) f'(z^{(n_l)}) W^{(n_l-1)} \frac{\partial}{\partial W_{ij}^{(l)}} a^{(n_l-1)} \\&= \underbrace{(a^{(n_l)} - y) f'(z^{(n_l)}) W^{(n_l-1)}}_{\delta^{(n_l)}} \frac{\partial}{\partial W_{ij}^{(l)}} z^{(n_l-1)} \\&\quad \underbrace{\hspace{10em}}_{\delta^{(n_l-1)}}\end{aligned}$$

BACK PROPOGATION

1. Perform forward propagation.
2. Compute δ for the output unit(s), e.g. $\delta = y - t$ in the case of MSE.
3. Backpropagate the δ 's using

$$\delta_j = f'(a_j) \sum_k w_{kj} \delta_k$$

for each hidden unit in the network.

4. Compute the required derivatives using

$$\frac{\partial L}{\partial w_{ji}^{(k)}} = \delta_j z_i.$$

BACK PROPOGATION

WHAT WAS WRONG WITH BACKPROPAGATION IN 1986?



1. Our labeled datasets were thousands of times too small.
2. Our computers were millions of times too slow.
3. We initialized the weights in a stupid way.
4. We used the wrong type of non-linearity.

XAVIER INITIALIZATION

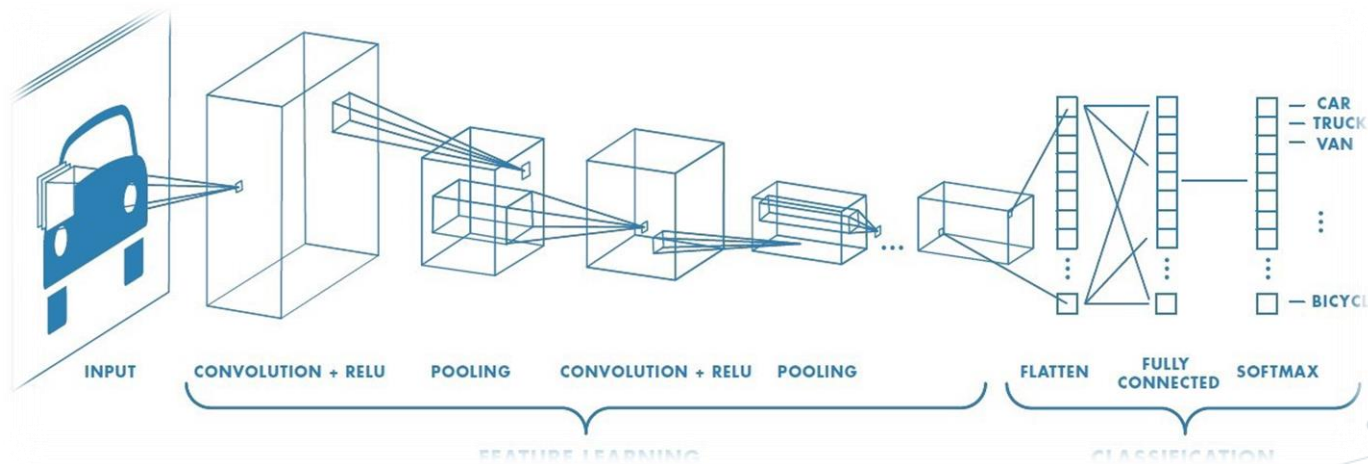
- Initialize weights using

$$w^{(l)} \sim \mathcal{N} \left(0, \sqrt{\frac{2}{n^{(l)} + n^{(l-1)}}} \right),$$

where $n^{(l)}$ is the number of neurons in layer l .

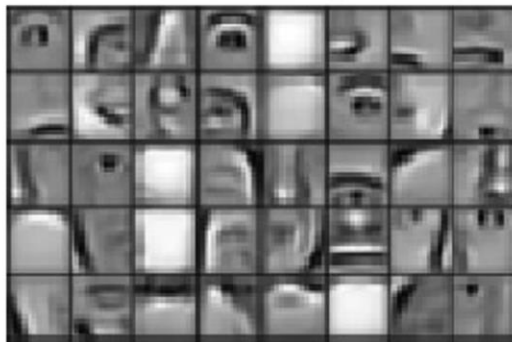
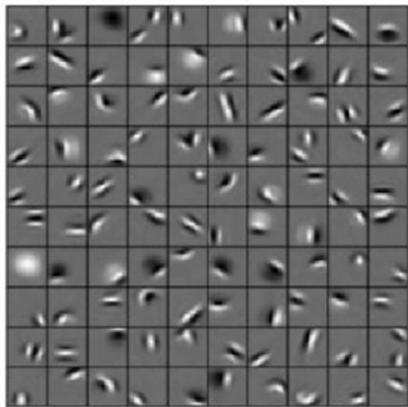
- This makes the variance of the activations in each layer similar to one another.

CONVOLUTIONAL NEURAL NETS



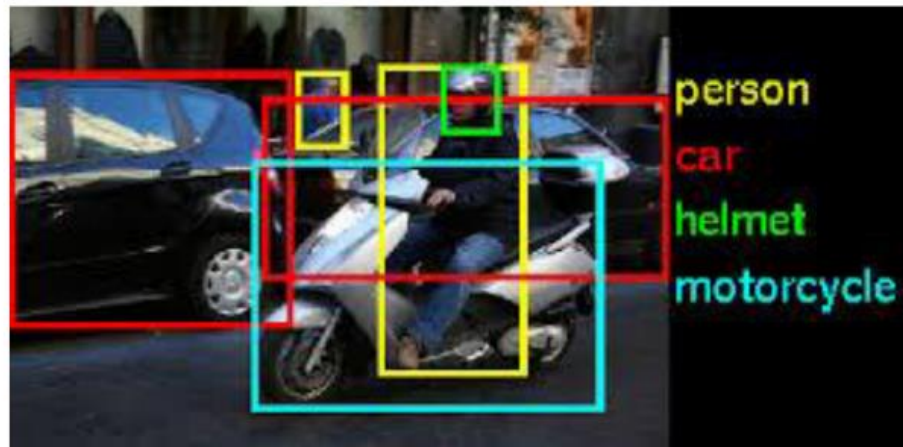
WHAT IS CNN?

- Category of neural networks particularly effective for image classification
- Deep CNNs extract a hierarchy of features from the input:
 - Lower layers extract local features such as lines and curves
 - Higher layers extract composites of local features; e.g. parts of a face



WHAT IS CNN?

IMAGE RECOGNITION



Too many parameters.

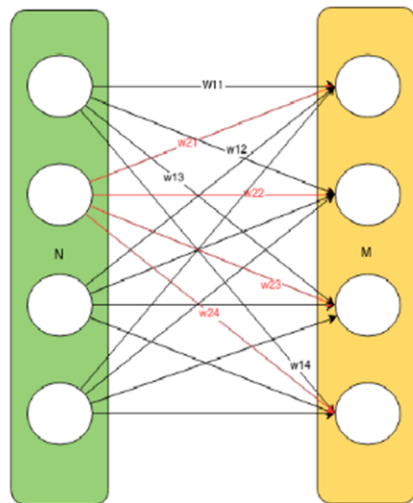
- $(\text{MNIST } 28 \times 28 \text{ pixels}) * (100 \text{ features}) = 78,400 \text{ params}$

Translation invariance.

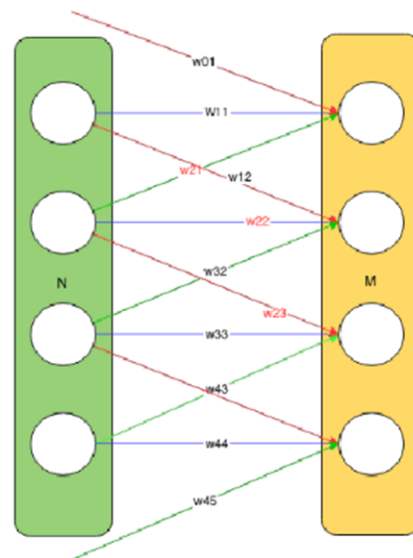
- Statistics of a patch (usually) does not depend on its location.

WHAT IS CNN?

SHARED WEIGHTS



**Fully-Connected
(Dense)**

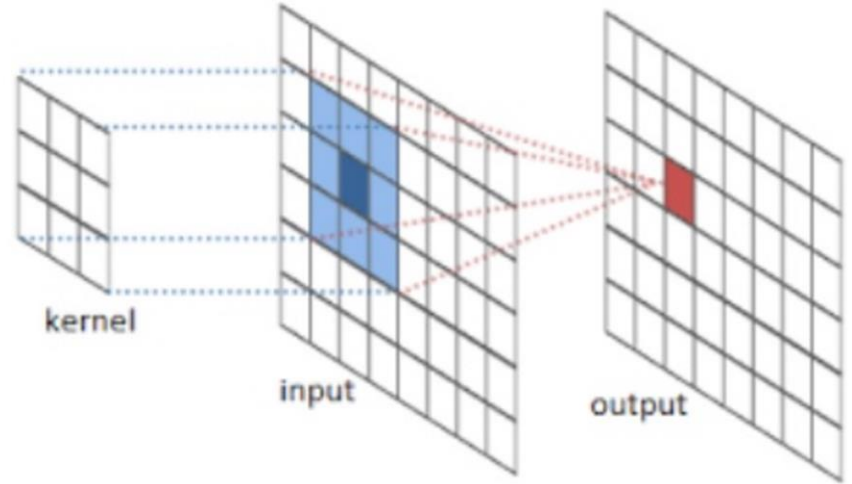


**Locally-Connected
with Shared Weights**

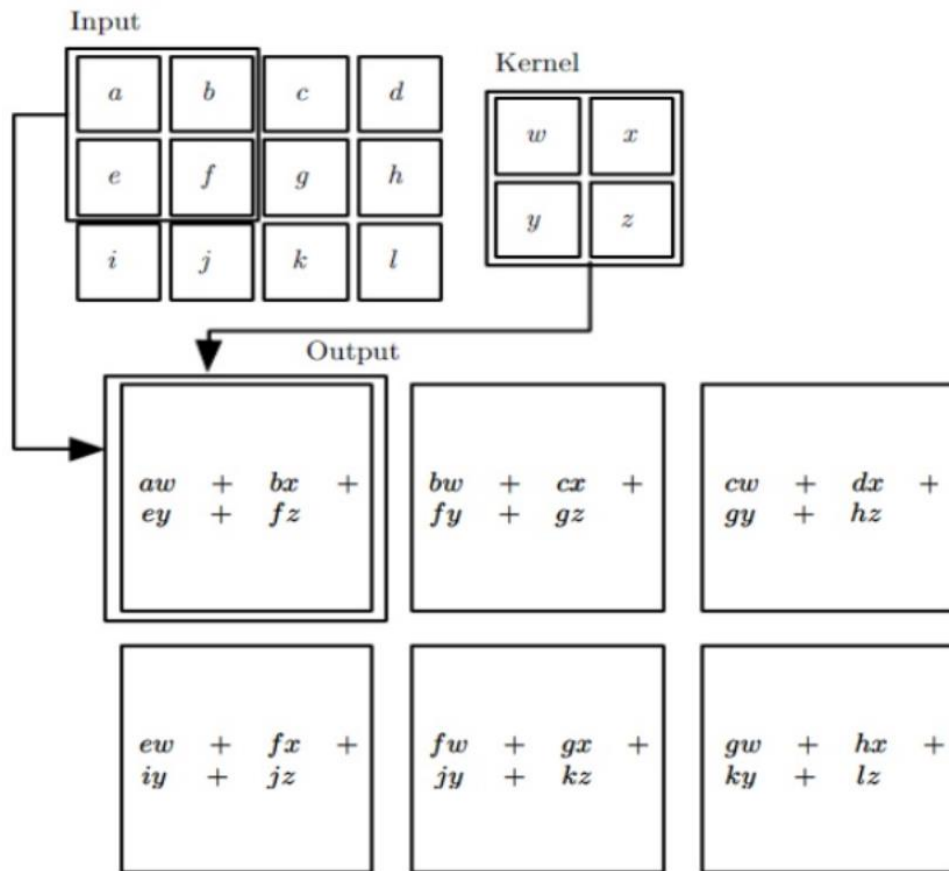
ALGORITHM

Convolutional networks are simply neural networks that use convolution in place of general matrix multiplication in at least one of their layers.

- Convolution is a mathematical operation having a linear form



ALGORITHM



ALGORITHM

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

6 x 6



*

1	0	-1
1	0	-1
1	0	-1

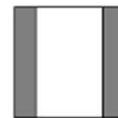
3 x 3



=

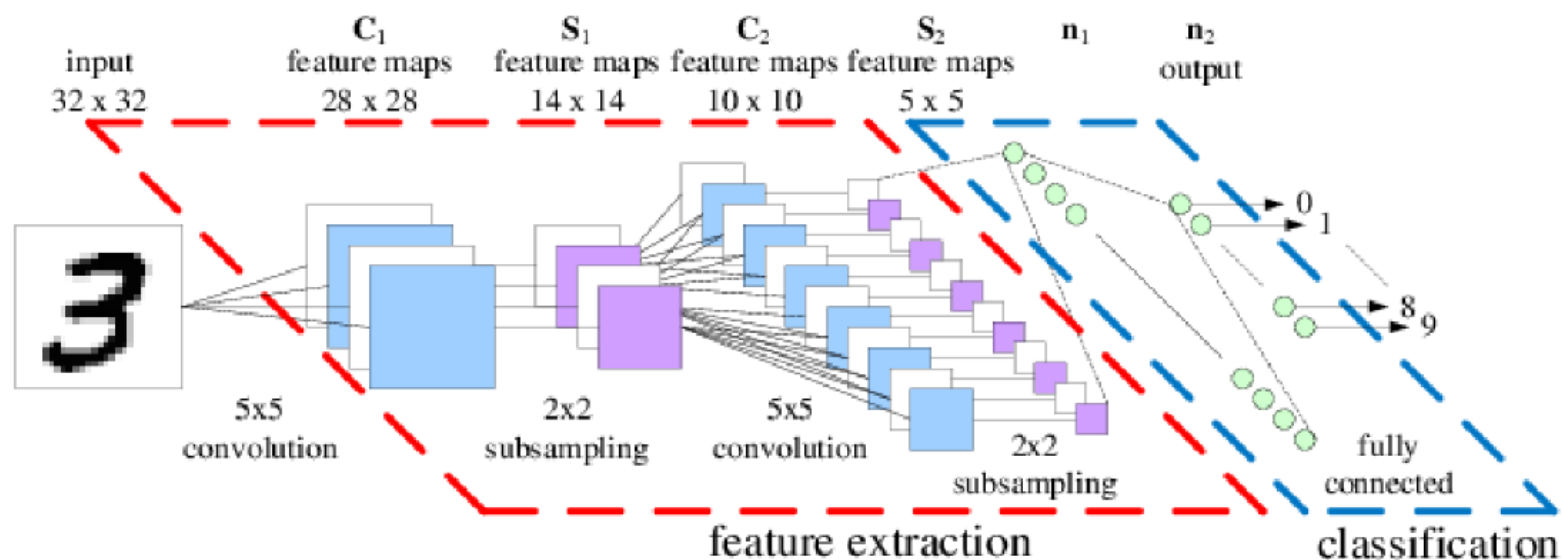
-0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

4 x 4



ARCHITECTURE

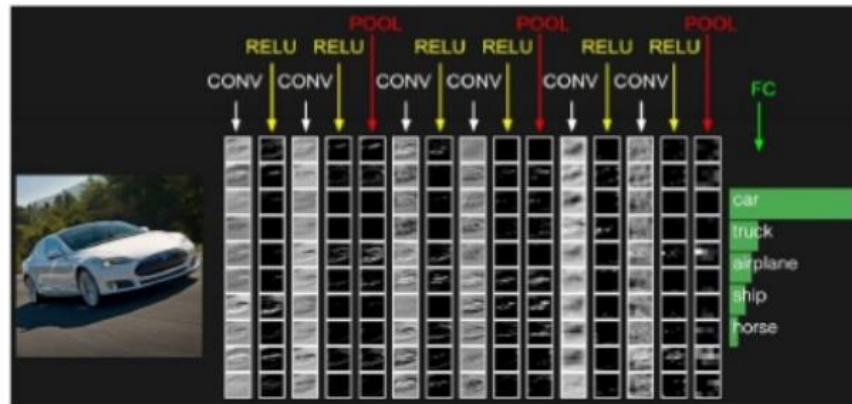
CNN architecture (e.g. LeNet)



ARCHITECTURE

- **INPUT** $[32 \times 32 \times 3]$ will hold the raw pixel values of the image, in this case an image of width 32, height 32, and with three color channels R,G,B.
- **CONV** layer will compute the output of neurons that are connected to local regions in the input, each computing a dot product between their weights and a small region they are connected to in the input volume. This may result in volume such as $[32 \times 32 \times 12]$ if we decided to use 12 filters.
- **RELU** layer will apply an elementwise activation function, such as the $\max(0, x)$ thresholding at zero. This leaves the size of the volume unchanged ($[32 \times 32 \times 12]$).
- **POOL** layer will perform a downsampling operation along the spatial dimensions (width, height), resulting in volume such as $[16 \times 16 \times 12]$.
- **FC** (i.e. fully-connected) layer will compute the class scores, resulting in volume of size $[1 \times 1 \times 10]$, where each of the 10 numbers correspond to a class score, such as among the 10 categories of CIFAR-10. As with ordinary Neural Networks and as the name implies, each neuron in this layer will be connected to all the numbers in the previous volume.

Also known as subsampling.
Max-pooling takes the maximum of a two by two square

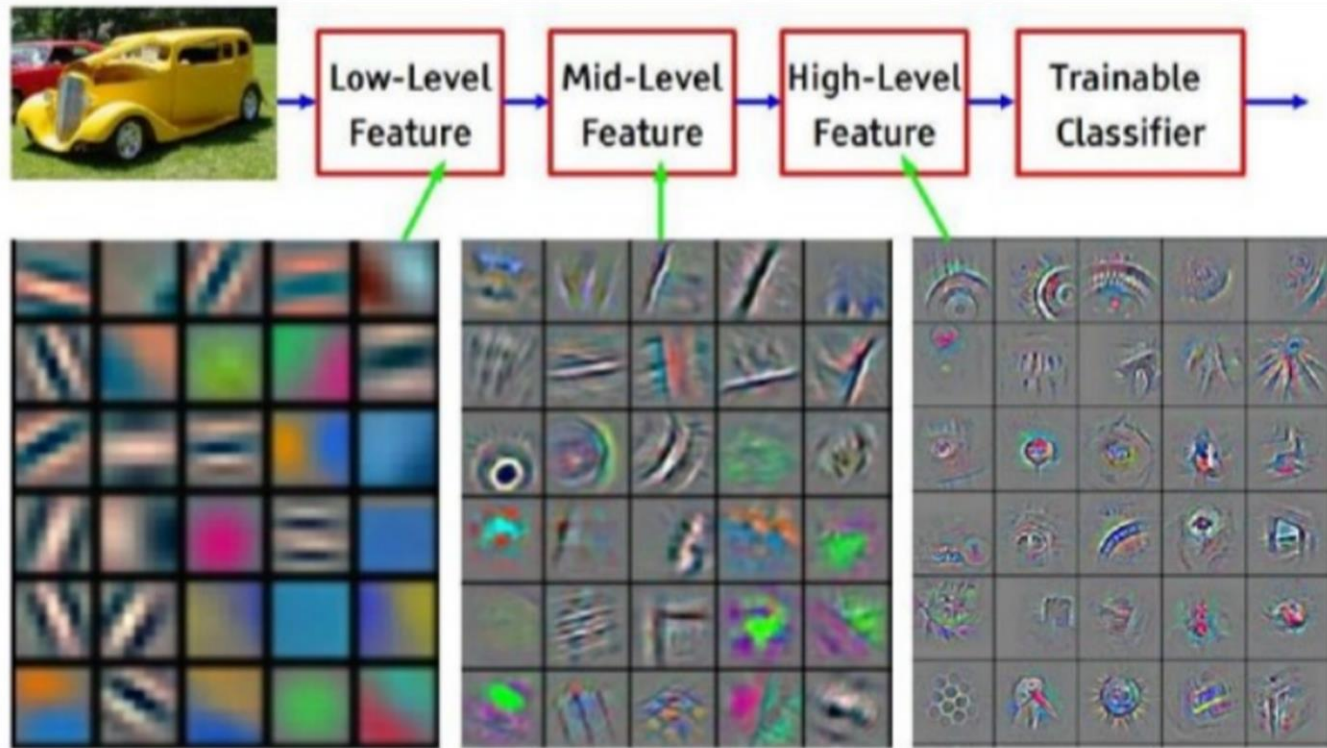


ALGORITHM

- Example:

- Consider an image $32 \times 32 \times 3$ and a $5 \times 5 \times 3$ filter.
- The convolution happens between a $5 \times 5 \times 3$ chunk of the image with the filter: $w^T x + b$
- In this example we get 75 dimensional vector and a bias term
- In this example, with a stride of 1, we get $28 \times 28 \times 1$ activation for 1 filter without padding
- If we have 6 filters, we would get $28 \times 28 \times 6$ without padding

FEATURE REPRESENTATION



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

PADDING

- The spatial (x, y) extent of the output produced by the convolutional layer is less than the respective dimensions of the input (except for the special case of 1 x 1 filter with a stride 1).
- As we add more layers and use larger strides, the output surface dimensions keep reducing and this may impact the accuracy.
- Often, we may want to preserve the spatial extent during the initial layers and downsample them at a later time.
- Padding the input with suitable values (padding with zero is common) helps to preserve the spatial size

PADDING

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

7x7 output!

in general, common to see CONV layers with stride 1, filters of size $F \times F$, and zero-padding with $(F-1)/2$. (will preserve size spatially)

e.g. $F = 3 \Rightarrow$ zero pad with 1

$F = 5 \Rightarrow$ zero pad with 2

$F = 7 \Rightarrow$ zero pad with 3

MAX POOLING

Single depth slice

x

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

y

max pool with 2x2 filters
and stride 2

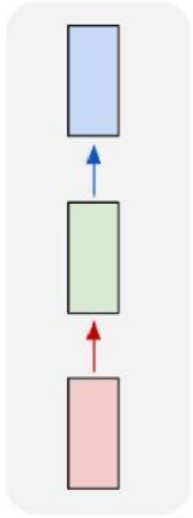
6	8
3	4

RECURRENT NEURAL NETS



TYPICAL NEURAL NETWORK.

one to one



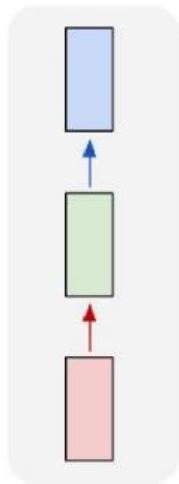
Vanilla Neural Networks

RNN

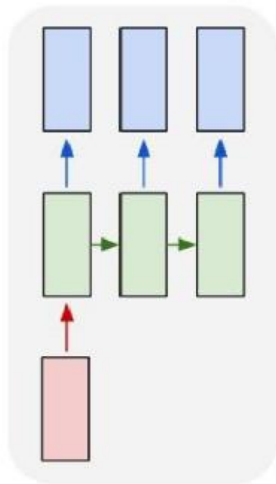
Primarily used to analyze sequential data, where sequence is important to the input, such as sentences, videos, etc.

RNN

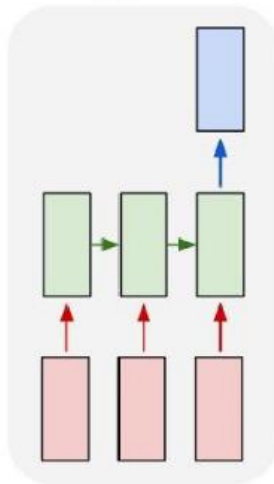
one to one



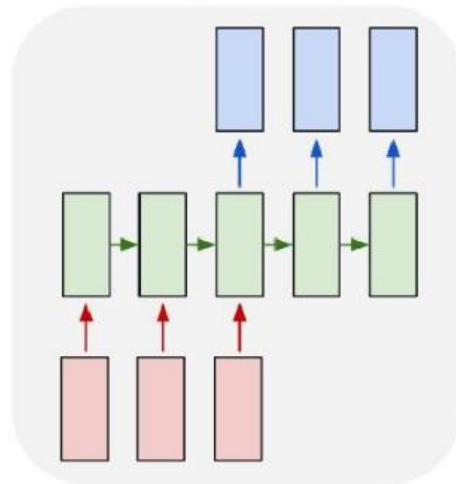
one to many



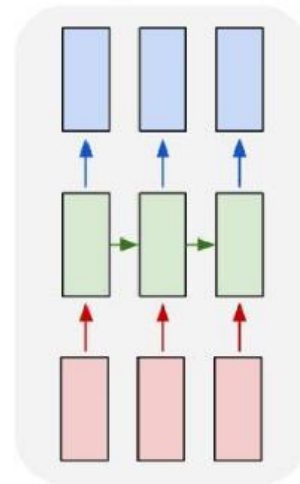
many to one



many to many



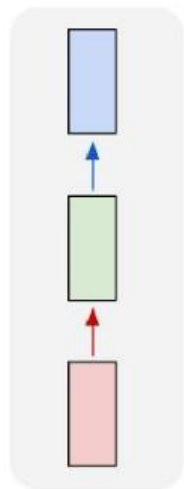
many to many



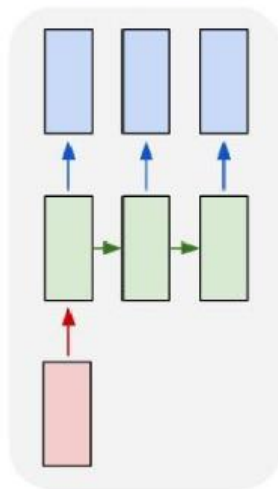
↖ e.g. **Image Captioning**
image → sequence of words

RNN

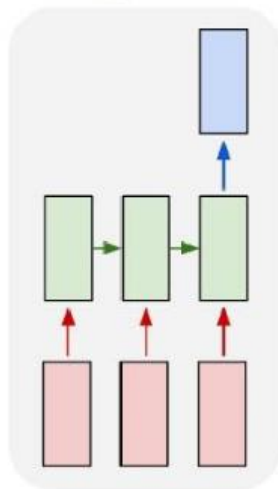
one to one



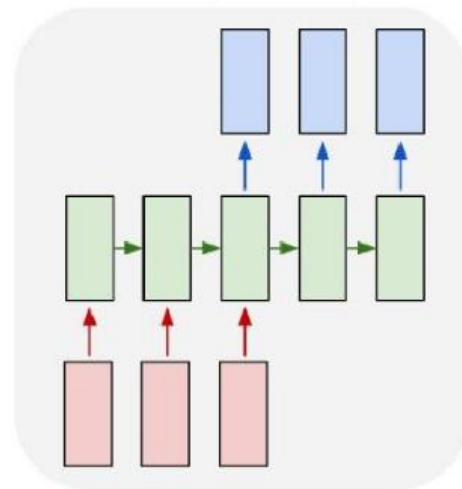
one to many



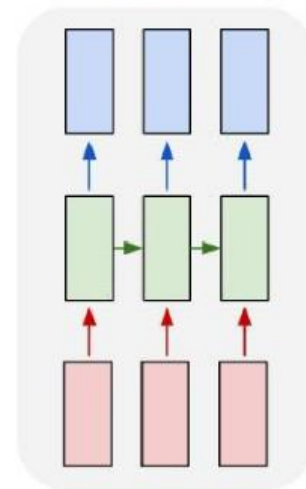
many to one



many to many



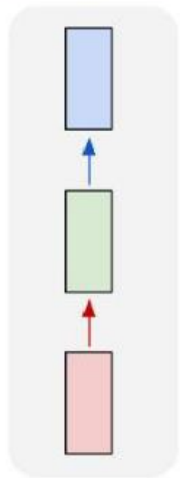
many to many



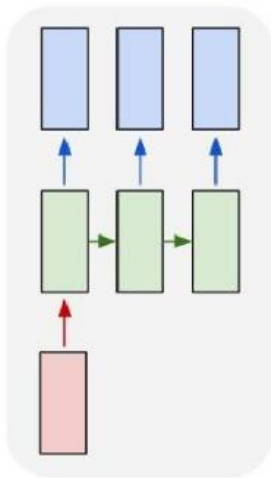
e.g. **Sentiment Classification**
sequence of words -> sentiment

RNN

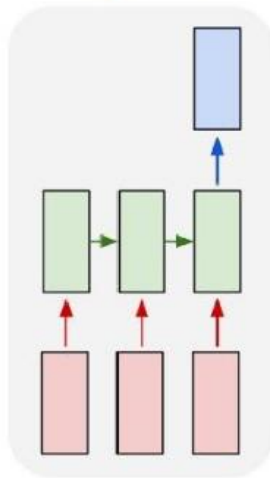
one to one



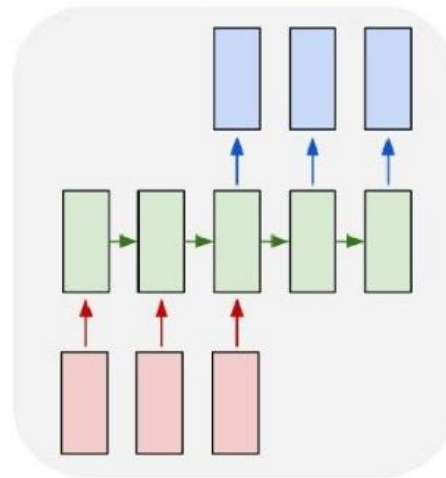
one to many



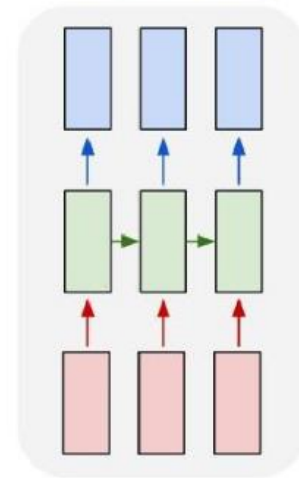
many to one



many to many



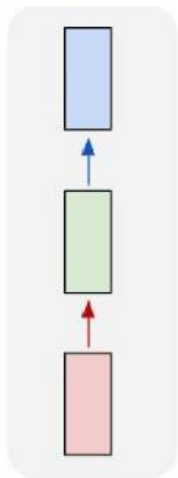
many to many



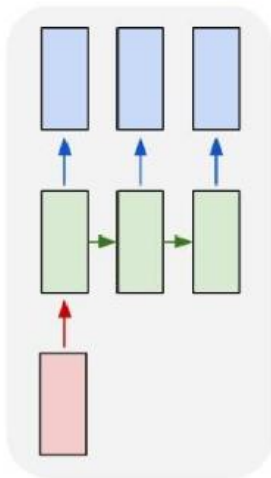
↖ e.g. **Machine Translation**
seq of words → seq of words

RNN

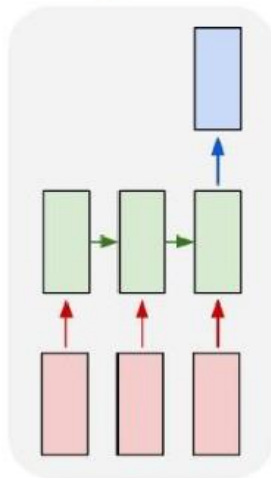
one to one



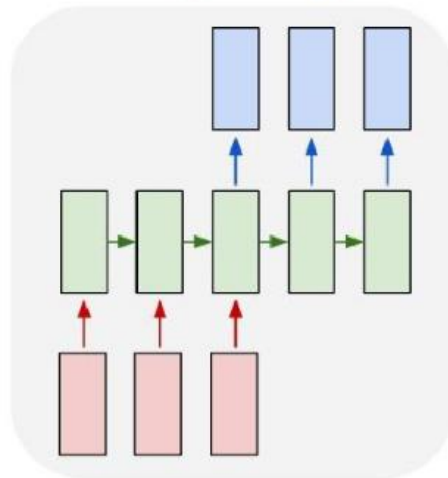
one to many



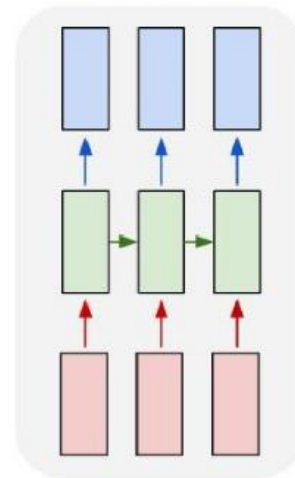
many to one



many to many



many to many



e.g. Video classification on frame level

